

Model V6.2 Against YOLO11

A purpose-built dense-mask segmenter reaches **+0.10 mask mAP50-95** over a COCO-pretrained YOLO11l-seg on the same boards, with 4× fewer parameters — and the gap sits exactly where the architecture predicts it should.

Status training in progress, epoch 23 of 120 **Hardware** RTX PRO 1000 Blackwell, 8 GB

Initialisation from scratch, no pretrained weights **Compiled** 2 September 2026

MASK MAP50-95

0.781

YOLO11l-seg 0.683

MASK MAP50

0.951

YOLO11l-seg 0.964

PARAMETERS

6.42_M

YOLO11l-seg ~27.6 M

RECALL

0.957

YOLO11l-seg 0.912

MIN / EPOCH

41.8

YOLO11l-seg 74.9

01 What the model does

It finds and outlines every component pad on a printed circuit board, one mask per physical object, and labels each with its shape class. Not a bounding box around a pad — the actual pixel boundary of that pad, separated from the pad touching it.

Four classes, taken from the annotation set: **Rectangle**, **Rectangle_concave**, **circle** (an annulus) and **circle_full** (a filled disc). Input is letterboxed to 512×512 ; instance reasoning happens on a 256×256 grid and the class map is produced at full resolution.

The data it learns from

SPLIT	IMAGES	INSTANCES	OBJECTS / IMAGE	MEDIAN OBJECT Ø
train	4,680	151,724	32.4	21.0–35.7 px
val	1,048	48,388	46.2	17.5–33.3 px
test	432	33,472	77.5	16.0–21.9 px

Split_Data, measured from the prepared arrays. Object size is the median equivalent diameter in pixels after letterboxing.

Two properties of this dataset shape every design decision that follows. **The objects are small** — a median `circle_full` is 16 pixels across in the test split, so a one-pixel error around its rim costs roughly a fifth of its IoU. And **the test split is not the training split**: its objects are 0.48–0.76× the linear size of training objects and sit 2.4× more densely per image. A model that only ever saw training-scale objects would meet something different at inference.

02 How it works

A shared encoder feeds two independent decoders, which feed four supervised heads. There are no bounding boxes anywhere in the design; an instance is a set of pixels that agree on where their centre is.

Semantic head

$512 \times 512 \times 5$

Per-pixel class. Trained with focal + Dice + Lovász-Softmax, the last optimising IoU directly rather than as a proxy.

Centre head

$256 \times 256 \times 4$

One Gaussian peak per object, per class. CenterNet focal loss. Peaks after non-maximum suppression become instance seeds.

Offset head

$256 \times 256 \times 2$

Each foreground pixel predicts the vector to its own object's centre. Pixels are then assigned to the seed they point at.

Boundary head

$512 \times 512 \times 1$

Supervised on instance-ID transitions. Splits candidates that the centre grouping merged, and sharpens the class map's edges.

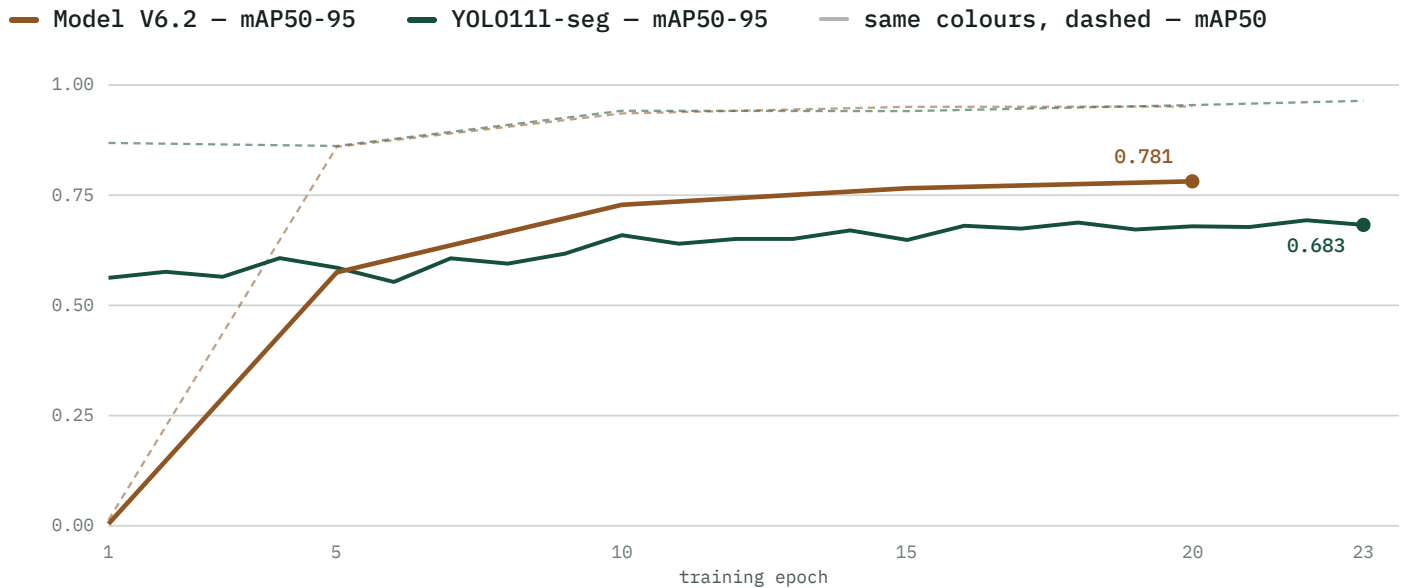
The encoder borrows YOLO's C3k2/SPPF blocks, the neck is a weighted bidirectional feature pyramid, and the instance grouping is the centre + offset scheme from Panoptic-DeepLab. 6,417,683 parameters across 464 layers, 42.75 GMAC per image.

Augmentation matched to the measured domain gap

Because the test split is smaller and denser than the training split, the training pipeline manufactures that shift: a 95-mode rotation and photometric cycle, all eight dihedral transforms (every class here is symmetric under them, so they are exactly label-preserving), a zoom-out ladder from 5% to 50% in 5% steps padded with black, and copy-paste augmentation that lifts `Rectangle_concave` from 0.47% of objects to roughly a quarter of the objects the model sees.

03 Where it stands against YOLO11l-seg

A YOLO11l-seg baseline was trained on these boards at the same 512 px input: COCO-pretrained, 23 epochs, 28.7 hours. Model V6.2 is currently at epoch 23 of 120, from random initialisation. Both curves below are mask mAP averaged over IoU 0.50 to 0.95 in steps of 0.05.



Solid lines are mAP50-95, the metric that rewards mask precision. Dashed lines are mAP50, which only asks whether the object was found. The two dashed lines nearly coincide; the two solid lines do not. That separation is the whole finding.

METRIC	MODEL V6.2	YOLO11L-SEG	DIFFERENCE
mask mAP50-95	0.7814	0.6827	+0.0987
mask mAP50	0.9513	0.9643	-0.0130
mask precision	0.8820	0.9543	-0.0723
mask recall	0.9567	0.9120	+0.0447
parameters	6,417,683	~27,600,000	4.3× fewer
minutes / epoch	41.8	74.9	1.8× faster

Mask metrics at each model's latest evaluation. V6.2 at epoch 20 (256 validation images); YOLO11l-seg at epoch 23 (its full validation split, Ultralytics' own metric implementation).

The two models fail differently, and that is the point. They agree almost exactly on whether an object is there — mAP50 differs by 0.013. They disagree by 0.099 on how well the mask fits once you demand IoU above 0.5.

YOLO finds the pad. V6.2 draws it.

04 Why the gap sits at high IoU

Three structural differences explain it, and all three are properties of the architectures rather than of how long either was trained.

Prototype masks are low-rank; dense masks are not

YOLO-seg predicts a small basis of prototype masks at reduced resolution and reconstructs each instance as a weighted combination of them. That is efficient and works well on large, distinct objects. Every mask it can express, though, lives in that low-dimensional span. On a 16-pixel disc, the rim is most of the object, and the rim is what the basis cannot spend capacity on. V6.2 predicts a class per pixel at full 512 px resolution; nothing constrains one mask to resemble a blend of others.

A mask cropped to a box inherits the box's error

YOLO-seg multiplies its mask by the detected bounding box. A box that is slightly loose admits background; slightly tight, and it shaves the object. At IoU 0.5 that rarely changes the verdict. At IoU 0.85 it decides it. V6.2 produces no boxes at all — an instance is a connected set of pixels whose predicted offsets agree on a centre — so there is no box error to inherit.

Axis-aligned boxes are a poor description of a rotated pad

These pads sit at arbitrary angles; the training cycle alone sweeps 45 distinct rotations. An axis-aligned box around a diagonal elongated pad is mostly background, which weakens both the mask crop and the non-maximum suppression that decides which detections survive. Dense per-pixel grouping has no orientation preference to lose.

A fourth difference is supervision rather than structure: V6.2 has an explicit boundary head trained on instance-ID transitions. Nothing in YOLO-seg is supervised on where one object stops and the next begins, and at 77 objects per test image, plenty of them touch.

The evidence within V6.2 agrees

CLASS	SHARE OF OBJECTS	AP50	AP50-95	GAP
Rectangle_concave (largest)	0.7%	0.968	0.833	0.135
Rectangle	25%	0.961	0.745	0.216
circle	14%	0.950	0.723	0.227
circle_full (smallest)	60%	0.863	0.612	0.251

Per-class average precision, Model V6.2 at epoch 20. The classes are ordered by object size.

The AP50-95 ordering is the size ordering, precisely inverted. The rarest class, `Rectangle_concave` — 680 training polygons — scores **best** at high IoU because it is large. The most common class scores worst because it is small. Mask geometry, not detection difficulty, is what separates them, which is the same conclusion the YOLO comparison reaches from the outside.

05 What this does not yet prove

The comparison above is indicative, not decisive. Five things stand between it and a claim that would survive review.

UNVERIFIED

The two models may not share a validation split. The YOLO run referenced a `Split_Data_yolo` directory that no longer exists. Until both are scored on the same images, the difference includes an unknown amount of split effect.

UNVERIFIED

Two different mAP implementations. Ultralytics computes its own; V6.2 computes its own. Both follow the COCO convention, but they have not been checked against each other on identical inputs.

OPEN

Neither model is converged. YOLO stopped at 23 of a planned 300 epochs; V6.2 is at 23 of 120. Both numbers will move, and not necessarily by the same amount.

OPEN

No error bars. With 432 test images, the difference between 0.78 and 0.80 may well be inside the sampling noise. A 1,000-fold bootstrap is needed; overlapping intervals would mean nothing has been beaten.

KNOWN

The labels cap what any model can score. 434 training images and 19.4% of the *test* split contain polygons that overwrite one another — 104 training instances erased outright. That is a ceiling on achievable IoU that no architecture removes.

One further asymmetry is worth stating plainly rather than burying: YOLO arrived COCO-pretrained, with a large visual prior. V6.2 started from random weights and 4,680 images. That makes the current gap more surprising, not less — but it also means V6.2 carries no transferable prior to a different board type, where YOLO would.

06 What remains

STEP	COST	EXPECTED
Finish the run (epoch 120)	~68 h	-
Fit the five decoder thresholds on validation	hours	+0.02-0.05
Dihedral test-time augmentation	hours	+0.01-0.03
Re-run both models under one protocol, with bootstrap CIs	1 day	the honest number
Repair the overlapping polygon labels	days	raises the ceiling
Replace centre+offset with distance-transform watershed	~3 days	cuts 22.6% of compute

Remaining work, ordered by expected return per unit of effort.
Estimated gains are the project's own prior estimates, not
measurements.



The first two require no retraining at all: they operate on the finished checkpoint and target precision directly, which is the one axis on which YOLO currently leads. The fourth is what turns this report from indicative into decisive.

Model V6.2 · 6,417,683 parameters · trained from random initialisation on 4,680 images, no pretrained weights, nothing downloaded.
Figures for Model V6.2 are measured on 256 validation images at epoch 20; figures for YOLO11l-seg are read from its own results.csv at epoch 23. Training was still in progress when this report was compiled.